```
# => #<ActiveSupport::TimeZone:0x7f73654d4f38 @utc_offset=nil,
@name="Madrid", ...>

# In Barcelona, 2010/03/28 02:00 +0100 becomes 2010/03/28 03:00 +0200
due to DST.
t = Time.local(2010, 3, 28, 1, 59, 59)
# => Sun Mar 28 01:59:59 +0100 2010
t.advance(seconds: 1)
# => Sun Mar 28 03:00:00 +0200 2010
```

- If `since` or `ago` jump to a time that can't be expressed with `Time` a `DateTime` object is returned instead.

17.1.1 `Time.current`

Active Support defines `Time.current` to be today in the current time zone. That's like `Time.now`, except that it honors the user time zone, if defined. It also defines the instance predicates `past?`, `today?`, and `future?`, all of them relative to `Time.current`.

When making Time comparisons using methods which honor the user time zone, make sure to use `Time.current` instead of `Time.now`. There are cases where the user time zone might be in the future compared to the system time zone, which `Time.now` uses by default. This means `Time.now.to_date`may equal `Date.yesterday`.

17.1.2 `all_day`, `all_week`, `all_month`, `all_quarter` and `all_year`

The method `all_day` returns a range representing the whole day of the current time.

```
now = Time.current
# => Mon, 09 Aug 2010 23:20:05 UTC +00:00
now.all_day
# => Mon, 09 Aug 2010 00:00:00 UTC +00:00..Mon, 09 Aug 2010 23:59:59
UTC +00:00
```

Analogously, `all_week`, `all_month`, `all_quarter` and `all_year` all serve the purpose of generating time ranges.

```
now = Time.current
# => Mon, 09 Aug 2010 23:20:05 UTC +00:00
now.all_week
# => Mon, 09 Aug 2010 00:00:00 UTC +00:00..Sun, 15 Aug 2010 23:59:59
UTC +00:00
now.all_week(:sunday)
# => Sun, 16 Sep 2012 00:00:00 UTC +00:00..Sat, 22 Sep 2012 23:59:59
UTC +00:00
now.all_month
# => Sat, 01 Aug 2010 00:00:00 UTC +00:00..Tue, 31 Aug 2010 23:59:59
UTC +00:00
now.all_quarter
# => Thu, 01 Jul 2010 00:00:00 UTC +00:00..Thu, 30 Sep 2010 23:59:59
UTC +00:00
now.all_year
# => Fri, 01 Jan 2010 00:00:00 UTC +00:00..Fri, 31 Dec 2010 23:59:59
UTC +00:00
```

17.2 Time Constructors

Active Support defines `Time.current` to be `Time.zone.now` if there's a user time zone defined, with fallback to `Time.now`:

```
Time.zone_default
# => #<ActiveSupport::TimeZone:0x7f73654d4f38 @utc_offset=nil,
@name="Madrid", ...>
Time.current
# => Fri, 06 Aug 2010 17:11:58 CEST +02:00
```

Analogously to `DateTime`, the predicates `past?`, and `future?` are relative to `Time.current`.

If the time to be constructed lies beyond the range supported by `Time` in the runtime platform, usecs are discarded and a `DateTime` object is returned instead.